# 4. AR Vision

Enter the Docker container, and then execute the tutorial commands within Docker to launch the program.

## 4.1. Overview

Augmented Reality (AR) is a technology that ingeniously blends virtual information with the real world. It extensively employs various technical means—including multimedia, 3D modeling, real-time tracking and registration, intelligent interaction, and sensing—to simulate and apply computer-generated virtual information (such as text, images, 3D models, music, and video) onto the real world. These two types of information complement each other, thereby achieving an "augmentation" of the real world.

AR systems possess three prominent characteristics: ① the integration of information from both the real and virtual worlds; ② real-time interactivity; and ③ the placement of virtual objects within a three-dimensional spatial environment.

Augmented Reality technology encompasses various new techniques and methods, including multimedia, 3D modeling, real-time video display and control, multi-sensor fusion, real-time tracking and registration, and scene fusion.

## 4.2. Usage Instructions

When utilizing the AR examples, the camera's intrinsic parameters are mandatory; otherwise, the program cannot run (the factory-provided Docker image already includes the completed camera intrinsic calibration). The intrinsic parameter file is located in the same directory as the code.

### 4.2.1. Camera Intrinsic Calibration

To launch the camera, enter the following command within the Docker container terminal:

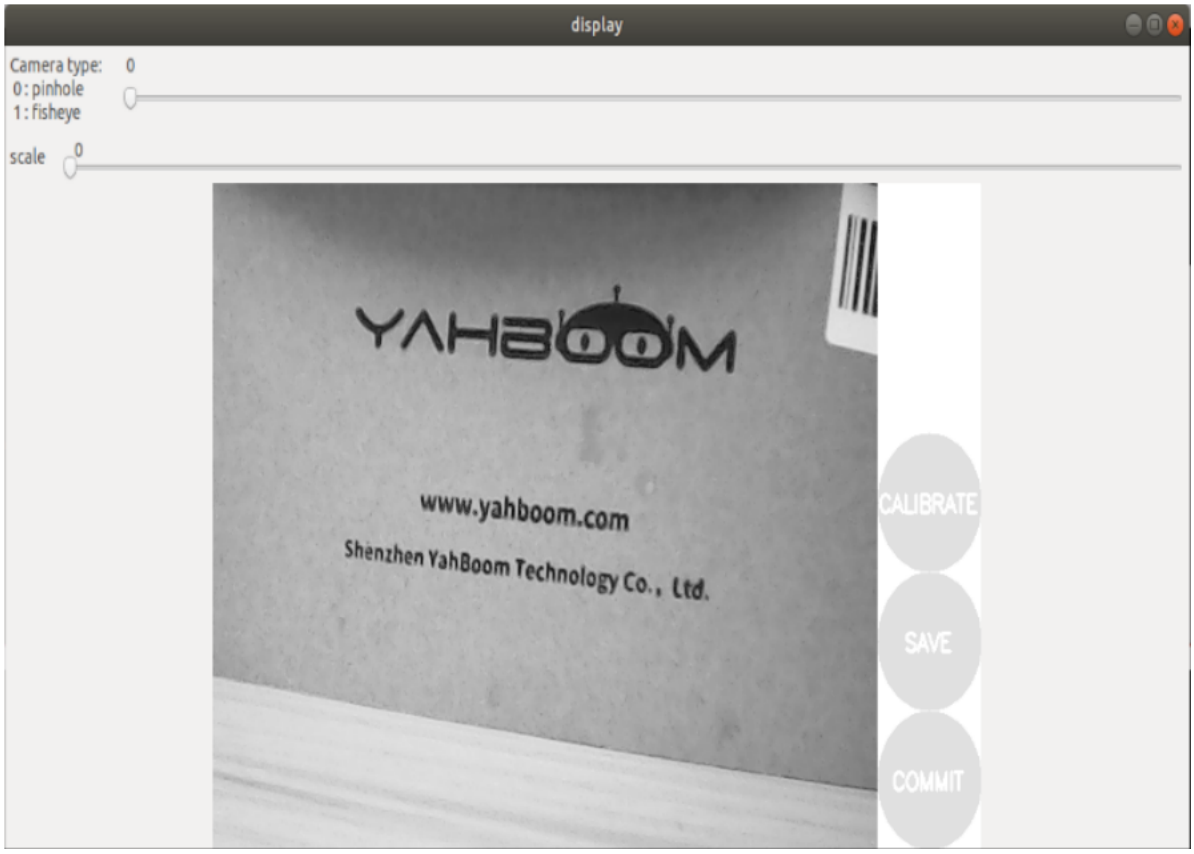
```
ros2 launch dofbot_info camera_arm_kin.launch.py
```

Launch the calibration node:

```
ros2 run camera_calibration cameracalibrator --size 9x6 --square 0.02 --ros-args
--remap /image:=/image_raw
```

`size`: The number of internal corners on the calibration chessboard pattern (e.g., `9x6`, indicating a total of six rows and nine columns of corners).

`square`: The side length of the chessboard squares, measured in meters. image: The image topic published by the camera




Calibration Interface

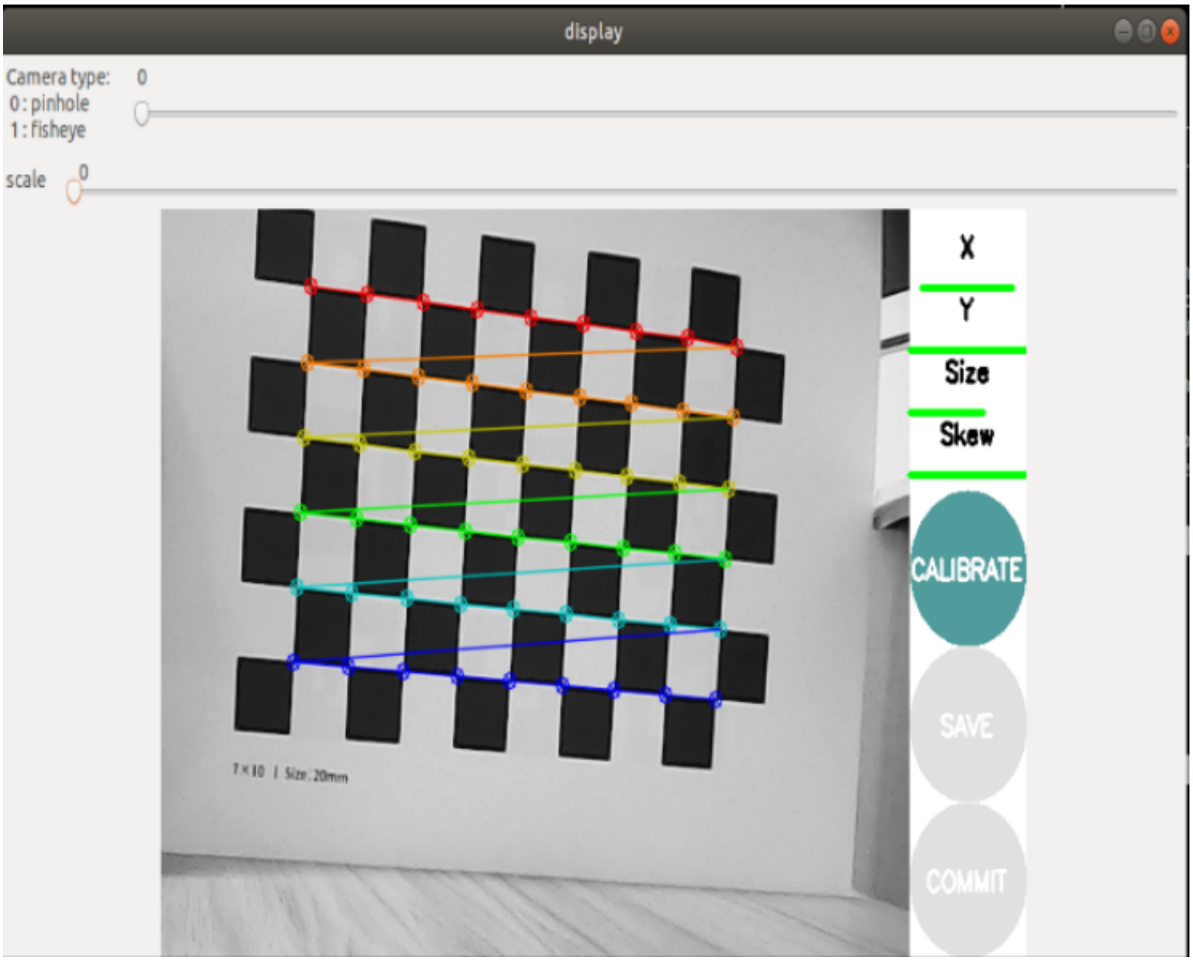
X: Left-to-right movement of the chessboard pattern within the camera's field of view

Y: Up-and-down movement of the chessboard pattern within the camera's field of view

Size: Forward-and-backward movement of the chessboard pattern within the camera's field of view

Skew: Tilting/rotational movement of the chessboard pattern within the camera's field of view

Once the application has launched successfully, place the chessboard pattern in the center of the frame and move it through various poses. The system will automatically detect the pattern. Ideally, as data is collected, the progress bars beneath [X], [Y], [Size], and [Skew] will transition from red to yellow, and finally to green, filling up as completely as possible.




- Click [CALIBRATE] to calculate the camera's intrinsic parameters. The more images collected, the longer this process will take; simply wait for it to complete. (60 to 70 images is usually sufficient; collecting too many may cause the system to freeze.)
- Click [SAVE] to save the calibration results to the file `/tmp/calibrationdata.tar.gz` on the currently active terminal session.

Once the calibration is finished, you can move the `/tmp/calibrationdata.tar.gz` file to another location to inspect its contents:

```
sudo mv /tmp/calibrationdata.tar.gz ~
```

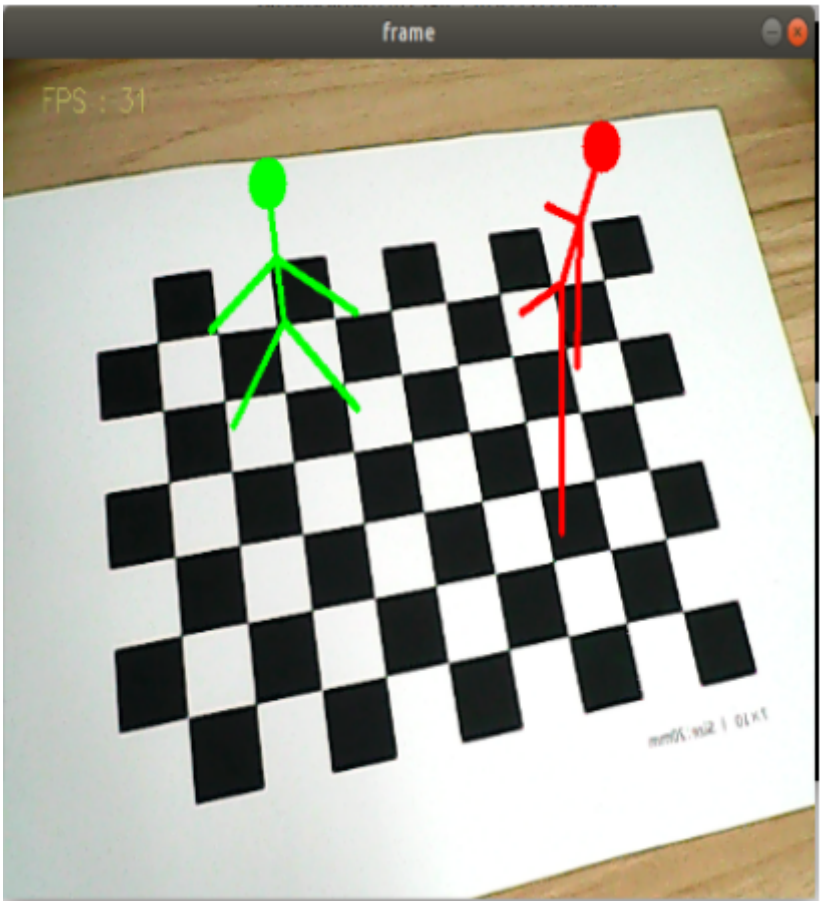
After extracting the archive, you will find the images captured during the recent calibration, along with an `ost.txt` file and an `ost.yaml` file. The `ost.yaml` file contains the camera's intrinsic parameters. Copy the contents of these intrinsic parameters and use them to overwrite the contents of the `astra.yaml` file located at `~/dofbot_ws/src/dofbot_visual/AR/`.

**Note:The factory-installed system image already comes with pre-calibrated parameters, allowing you to launch and use the program immediately without further calibration. If you have previously launched a separate camera preview window or process, please ensure you terminate that camera process *before* launching this calibration program.**

### 4.2.2. Program Startup

After accessing the Docker container terminal, enter the following command:

```
ros2 run dofbot_pro_vision simple_ar
```




Once the program has launched, place the chessboard pattern in front of the camera. Note that the *entire* chessboard pattern must be visible; otherwise, the program will terminate. Once the complete chessboard pattern is successfully recognized, the following 12 visual effects will be displayed:

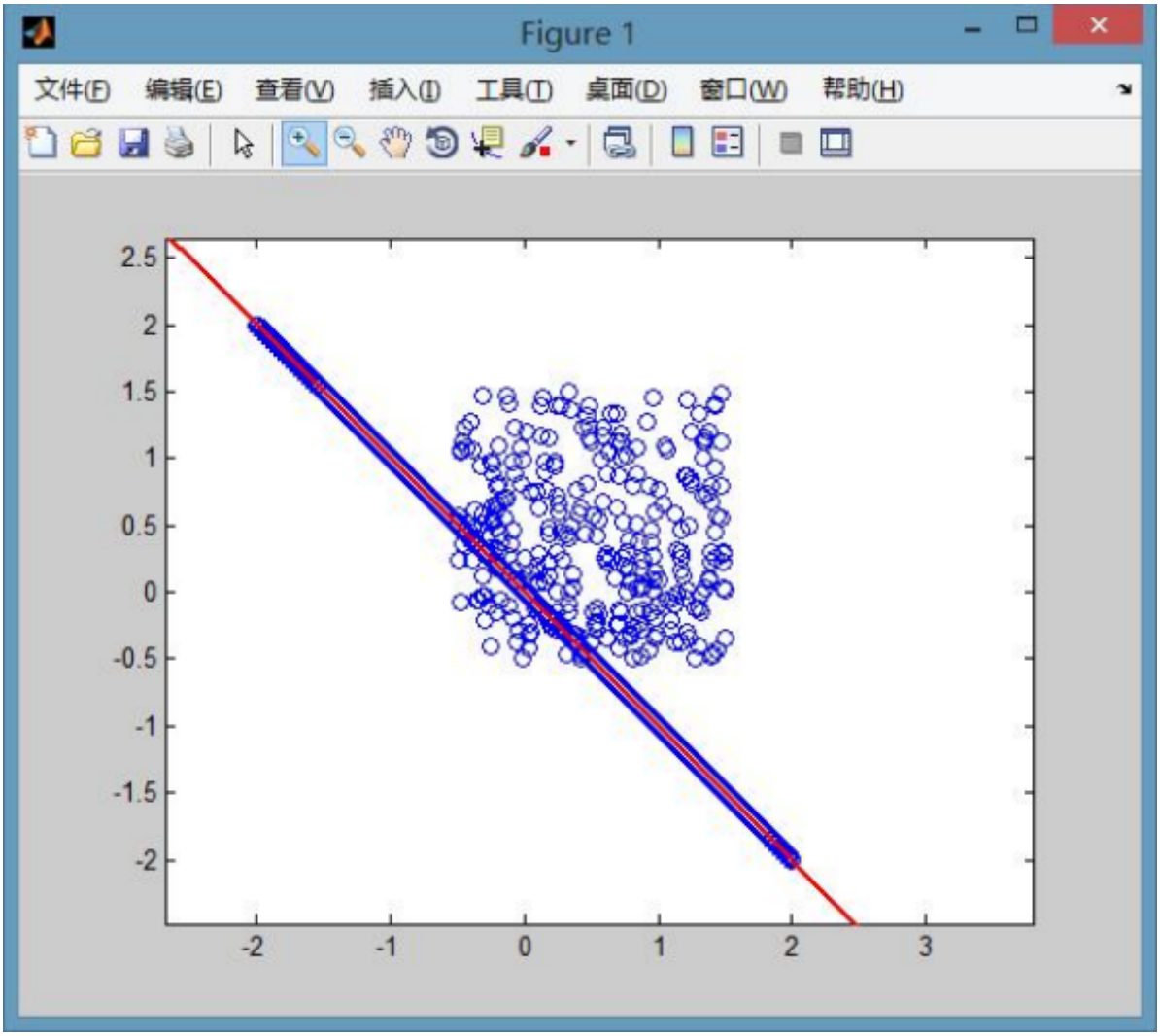
```
["Triangle", "Rectangle", "Parallelogram","WindMill","TableTennisTable",
"Ball","Arrow", "Knife", "Desk","Bench", "Stickman", "ParallelBars"]
```

Click on the image window and press the 'F' key to cycle through the different displayed effects.

### 4.3.1. RANSAC Approach

- **Algorithm Principle:** Utilizes the RANSAC approach to determine an object's pose based on 3D-to-2D point correspondences.

The RANSAC (Random Sample Consensus) algorithm is a classic method originally developed for data processing. Its purpose is to extract specific components of an object or dataset even when significant noise is present. The figure below illustrates the effectiveness of the RANSAC algorithm. In the diagram, a subset of points clearly aligns with a specific straight line, while another cluster of points constitutes pure noise. The objective is to determine the equation of the straight line amidst this heavy noise—in this particular example, the volume of noisy data is three times greater than that of the inlier data forming the line.




```
    Had the Least Squares method been applied instead, it would not have yielded
this result; the resulting line would likely have been positioned slightly higher
than the actual line shown in the figure.
```

- **Basic Assumptions of RANSAC:**
- The data consists of "inliers"—i.e., data points whose distribution can be explained by a specific set of model parameters.
- "Outliers" are data points that do *not* fit the established model.
- Any data points falling outside these two categories are classified as noise.
- **Causes of Outliers:** Outliers may arise from various sources, including: extreme values within the noise distribution; errors in measurement methodologies; or incorrect assumptions regarding the nature of the data. RANSAC also makes the following assumption: given a set of (typically small) inliers, there exists a procedure capable of estimating the model parameters—a model that can explain or fit these inliers.

### 4.3.2. Source Code

Path to the provided source code package:

```
/dofbot_pro_vision/dofbot_pro_vision/simple_ar.py
```

```
#common lib
import os
import sys
import time
```

```python
import cv2 as cv
import numpy as np
from cv_bridge import CvBridge

#ros2 lib
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from sensor_msgs.msg import CompressedImage,Image

print("import done")

cv_edition = cv.__version__
print("cv_edition: ",cv_edition)

class simple_AR(Node):
    def __init__(self,name):
        super().__init__(name)
        #create pub
        self.pub_img = self.create_publisher(Image,'/simpleAR/camera',1)
        #create sub
        self.sub_graphics =
self.create_subscription(String,'/Graphics_topic',self.choose_Graphics,1)

        self.flip = True
        self.index = 0
        self.graphics = ["Triangle", "Rectangle", "Parallelogram","WindMill",
                         "TableTennisTable", "Ball", "Arrow", "Knife", "Desk",
                         "Bench", "Stickman", "ParallelBars"]
        self.Graphics = self.graphics[self.index]
        self.axis = np.float32([
            [0, 0, -1], [0, 8, -1], [5, 8, -1], [5, 0, -1],
            [1, 2, -1], [1, 6, -1], [4, 2, -1], [4, 6, -1],
            [1, 0, -4], [1, 8, -4], [4, 0, -4], [4, 8, -4],
            [1, 2, -4], [1, 6, -4], [4, 2, -4], [4, 6, -4],
            [0, 1, -4], [3, 2, -1], [2, 2, -3], [3, 2, -3],
            [1, 2, -3], [2, 2, -4], [2, 2, -5], [0, 4, -4],
            [2, 3, -4], [1, 3, -4], [4, 3, -5], [4, 5, -5],
            [1, 2, -3], [1, 6, -3], [5, 2, -3], [5, 6, -3],
            [3, 4, -5], [0, 6, -4], [5, 6, -4], [2, 8, -4],
            [3, 8, -4], [2, 6, -4], [2, 0, -4], [1, 5, -4],
            [3, 0, -4], [3, 2, -4], [0, 3, -4], [1, 2, -4],
            [4, 2, -4], [5, 3, -4], [2, 7, -4], [3, 7, -4],
            [3, 3, -1], [3, 5, -1], [1, 5, -1], [1, 3, -1],
            [3, 3, -3], [3, 5, -3], [1, 5, -3], [1, 3, -3],
            [1, 3, -6], [1, 5, -6], [3, 3, -4], [3, 5, -4],
            [0, 0, -4], [3, 1, -4], [1, 1, -4], [0, 2, -4],
            [2, 4, -4], [4, 4, -4], [0, 8, -4], [5, 8, -4],
            [5, 0, -4], [0, 4, -5], [5, 4, -4], [5, 4, -5],
            [2, 5, -1], [2, 7, -1], [2, 6, -3], [2, 6, -5],
            [2, 5, -3], [2, 7, -3]
            ])
        self.frame = None
        self.img_name = 'img'
        self.patternSize = (6, 9)
        self.bridge = CvBridge()
        yaml_path = '/root/dofbot_ws/src/dofbot_pro_vision/config/camera.yaml'
```

```
        #yaml_path =
'/root/dofbot_pro_ws/src/dofbot_pro_vision/config/camera.yaml'
        if os.path.exists(yaml_path):
            fs = cv.FileStorage(yaml_path, cv.FileStorage_READ)
            self.cameraMatrix = fs.getNode("camera_matrix").mat()
            self.distCoeffs = fs.getNode("distortion_coefficients").mat()
        else:
            self.distCoeffs, self.cameraMatrix = (), ()
        self.objectPoints = np.zeros((6 * 9, 3), np.float32)
        self.objectPoints[:, :2] = np.mgrid[0:6, 0:9].T.reshape(-1, 2)
        self.capture = cv.VideoCapture(0)
        if cv_edition[0]=='3': self.capture.set(cv.CAP_PROP_FOURCC,
cv.VideoWriter_fourcc(*'XVID'))
        else: self.capture.set(cv.CAP_PROP_FOURCC, cv.VideoWriter.fourcc('M',
'J', 'P', 'G'))
        self.capture.set(6, cv.VideoWriter_fourcc('M', 'J', 'P', 'G'))
        self.capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
        self.capture.set(cv.CAP_PROP_FRAME_HEIGHT, 480)
        self.timer = self.create_timer(0.001, self.on_timer)


    def choose_Graphics(self,msg):
        if not isinstance(msg, String): return
        if msg.data in self.graphics: self.Graphics = msg.data
        else: self.graphics_update()

    def graphics_update(self):
        self.index += 1
        if self.index >= len(self.graphics): self.index = 0
        self.Graphics = self.graphics[self.index]

    def on_timer(self):
        ret, frame = self.capture.read()
        action = cv.waitKey(10) & 0xFF
        frame =self.process(frame, action)
        cv.imshow('frame', frame)
        #if len(binary) != 0: cv.imshow('frame', ManyImgs(1, ([frame, binary])))
        #else:cv.imshow('frame', frame)
        if action == ord('q') or action == 113:
            self.capture.release()
            cv.destroyAllWindows()

    def process(self, img, action):
        if self.flip == True: img = cv.flip(img, 1)
        if action == ord('f') or action == ord('F'): self.graphics_update()
        gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
        # 查找每个图片的角点
        # Find the corner of each image
        retval, corners = cv.findChessboardCorners(
            gray, self.patternSize, None,
            flags=cv.CALIB_CB_ADAPTIVE_THRESH + cv.CALIB_CB_NORMALIZE_IMAGE +
cv.CALIB_CB_FAST_CHECK)
        # 查找角点亚像素
        # Find corner subpixels
        if retval:
            corners = cv.cornerSubPix(
                gray, corners, (11, 11), (-1, -1),
                (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001))
```

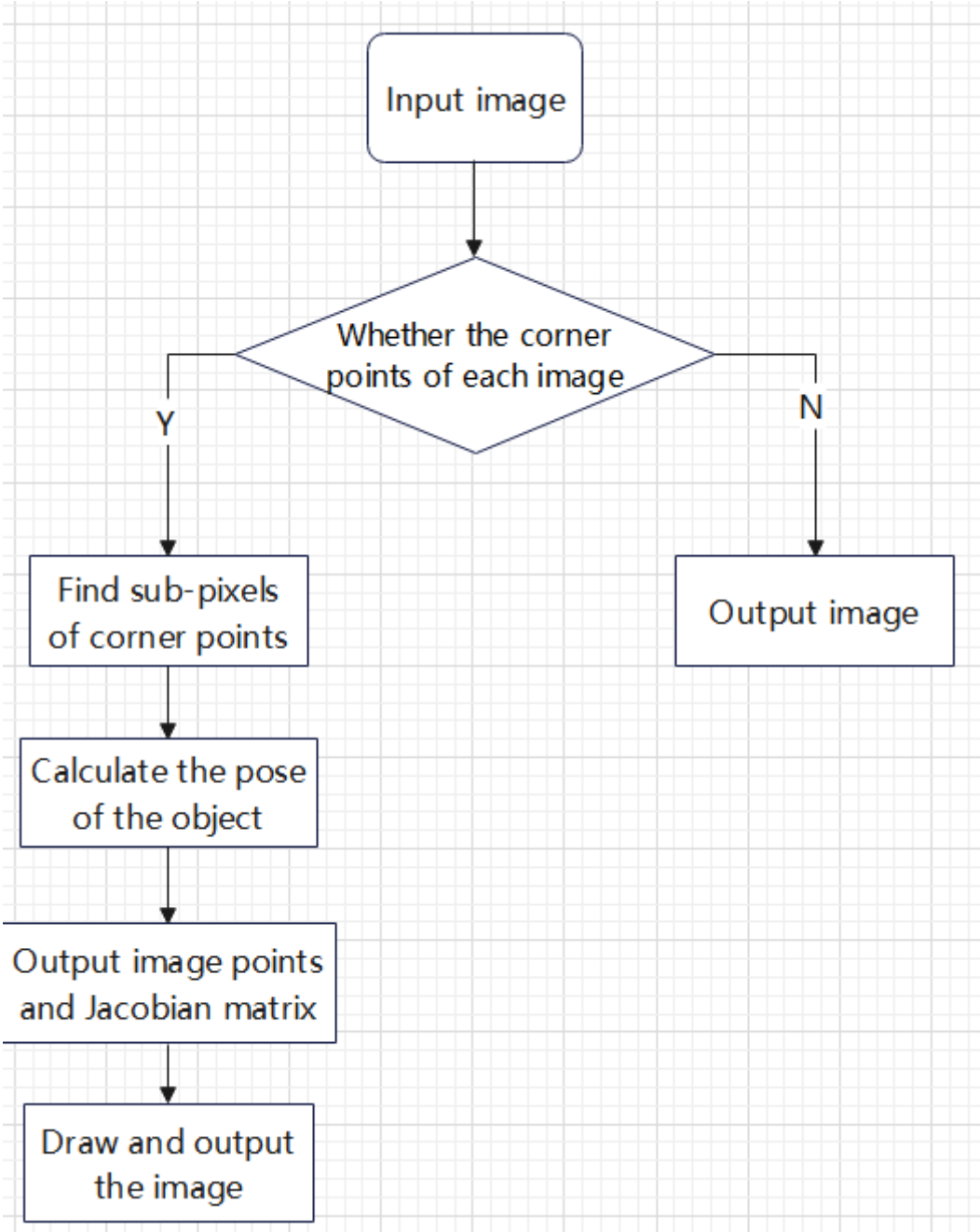
```
            # 计算对象姿态solvePnPRansac
            # Compute object pose solvePnPRansac
            retval, rvec, tvec, inliers = cv.solvePnPRansac(
                self.objectPoints, corners, self.cameraMatrix, self.distCoeffs)
            # 输出图像点和雅可比矩阵
            # Output image points and Jacobian matrix
            image_Points, jacobian = cv.projectPoints(
                self.axis, rvec, tvec, self.cameraMatrix, self.distCoeffs, )
            img = self.draw(img, corners, image_Points)
        cv.putText(img, self.Graphics, (240, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9,
(0, 0, 255), 1)
        self.pub_img.publish(self.bridge.cv2_to_imgmsg(img, "bgr8"))
        return img

    def draw(self, img, corners, image_Points):
        # drawContours函数中绘图颜色顺序是bgr
        # drawContours the color order of the drawing is BGR
        img_pts = np.int32(image_Points).reshape(-1, 2)
        if self.Graphics == "Triangle":
            cv.drawContours(img, [np.array([img_pts[14], img_pts[15],
img_pts[23]])], -1, (255, 0, 0), -1)
        elif self.Graphics == "Rectangle":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[13],
img_pts[15], img_pts[14]])], -1, (0, 255, 0), -1)
        elif self.Graphics == "Parallelogram":
            cv.drawContours(img, [np.array([img_pts[12], img_pts[10],
img_pts[15], img_pts[9]])], -1, (65, 105, 225), 1)
        elif self.Graphics == "WindMill":
            cv.drawContours(img, [np.array([img_pts[60], img_pts[38],
img_pts[61], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[10], img_pts[14],
img_pts[58], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[62], img_pts[63],
img_pts[23], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.drawContours(img, [np.array([img_pts[25], img_pts[64],
img_pts[65], img_pts[21]])], -1, (0, 0, 255), -1)
            cv.line(img, tuple(img_pts[64]), tuple(img_pts[35]), (0, 255, 0), 3)
        elif self.Graphics == "TableTennisTable":
            cv.line(img, tuple(img_pts[0]), tuple(img_pts[60]), (255, 0, 0), 3)
            for i in range(1, 4):
                cv.line(img, tuple(img_pts[i]), tuple(img_pts[65 + i]), (255, 0,
0), 3)
            cv.drawContours(img, [np.array([img_pts[60], img_pts[66],
img_pts[67], img_pts[68]])], -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[23], img_pts[69],
img_pts[71], img_pts[70]])], -1, (0, 0, 255), -1)
        elif self.Graphics == "Ball": cv.circle(img, tuple(img_pts[22]), 30, (0,
0, 255), -1)
        elif self.Graphics == "Arrow":
            cv.drawContours(img, [np.array([img_pts[13], img_pts[34],
img_pts[36]])], -1, (0, 255, 0), -1)
            cv.drawContours(img, [np.array([img_pts[37], img_pts[15],
img_pts[10], img_pts[38]])], -1, (0, 255, 0), -1)
        elif self.Graphics == "Knife":
            cv.drawContours(img, [np.array([img_pts[58], img_pts[24],
img_pts[35], img_pts[47]])], -1, (160, 252, 0),
                            -1)
```

```
            cv.drawContours(img, [np.array([img_pts[40], img_pts[38],
img_pts[21], img_pts[41]])], -1, (30, 144, 255),
                            -1)
            cv.drawContours(img, [np.array([img_pts[42:46]])], -1, (0, 0, 255),
-1)
        elif self.Graphics == "Desk":
            for i in range(4):
                cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(163, 148, 128), 3)
            cv.drawContours(img, [np.array([img_pts[14], img_pts[12],
img_pts[13], img_pts[15]])], -1, (0, 199, 140),
                            -1)
        elif self.Graphics == "Bench":
            for i in range(4):
                cv.line(img, tuple(img_pts[48 + i]), tuple(img_pts[52 + i]),
(255, 0, 0), 3)
            cv.drawContours(img, [img_pts[52:56]], -1, (0, 0, 255), -1)
            cv.drawContours(img, [img_pts[54:58]], -1, (139, 69, 19), -1)
        elif self.Graphics == "Stickman":
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[4]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[6]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[18]), tuple(img_pts[21]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[19]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[20]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[21]), tuple(img_pts[22]), (0, 0, 255), 3)
            cv.circle(img, tuple(img_pts[22]), 15, (0, 0, 255), -1)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[72]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[73]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[74]), tuple(img_pts[37]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[76]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[77]), (0, 255, 0), 3)
            cv.line(img, tuple(img_pts[37]), tuple(img_pts[75]), (0, 255, 0), 3)
            cv.circle(img, tuple(img_pts[75]), 15, (0, 255, 0), -1)
        elif self.Graphics == "ParallelBars":
            for i in range(4):
                cv.line(img, tuple(img_pts[4 + i]), tuple(img_pts[12 + i]),
(255, 0, 0), 3)
            cv.line(img, tuple(img_pts[8]), tuple(img_pts[9]), (0, 0, 255), 3)
            cv.line(img, tuple(img_pts[10]), tuple(img_pts[11]), (0, 0, 255), 3)
        return img

def main():
    print("start")
    rclpy.init()
    ar = simple_AR('simple_AR')
    rclpy.spin(ar)
```

Program Flowchart


Input image
Whether the corner points of each image
Y
N
Find sub-pixels of corner points
Output image
Calculate the pose of the object
Output image points and Jacobian matrix
Draw and output the image